Plan de Frontend — SoccerApi

> Fase 1: Web | Fase 2: Mobile (Flutter) Backend: Laravel 12 + Sanctum + MySQL Fecha: Julio 2026

---

1. Stack tecnológico (Fase 1 — Web)

Capa	Elección	Por qué
Framework	React + Vite + TypeScript	Sin overhead de SSR (no hace falta SEO en un dashboard autenticado). TS tipa los contratos de la API.
Estado de servidor	TanStack Query (React Query)	Cache, refetch, invalidación automática tras mutaciones (ej: predicción → invalidar leaderboard). Soporta polling con `refetchInterval` para scoring/leaderboard en vivo.
Estado de auth/sesión	Zustand + persist (localStorage)	Simple, sin boilerplate, ideal para token + usuario logueado.
Auth	Bearer token (Sanctum)	Mismo mecanismo reusable en Flutter (Fase 2), a diferencia de Sanctum SPA cookies que solo aplica a web.

---

2. Arquitectura de carpetas

```
src/
├── api/
│   ├── client.ts            # instancia fetch/axios con interceptor Bearer + manejo
422
│   ├── endpoints/
│   │   ├── auth.ts
│   │   ├── tournaments.ts
│   │   ├── stages.ts
│   │   ├── teams.ts
│   │   ├── matches.ts
│   │   ├── groups.ts
│   │   ├── predictions.ts
│   │   ├── invitations.ts
│   │   ├── pool.ts
│   │   ├── bracket.ts        # tournament-group, advancement-rule, advancements
│   │   └── subscriptions.ts
│   └── types.ts
```

```
├── hooks/
│   ├── useAuth.ts
│   ├── useTournaments.ts
│   ├── useMatches.ts
│   ├── usePredictions.ts
│   ├── useLeaderboard.ts
│   └── useBracket.ts
├── store/
│   └── authStore.ts          # Zustand: token, user, login(), logout()
├── lib/
│   └── errorHandler.ts       # normaliza errores 422 de Laravel → mensajes por
campo
├── pages/
│   ├── auth/ (Login, Register)
│   ├── tournaments/
│   │   ├── Bracket/ (BracketView, GroupStandings, KnockoutMatch)
│   │   └── admin/ (ApproveGroups, ConfirmAdvancements, ManageStages)
│   ├── groups/ (List, Create, Detail, Leaderboard, Scoring)
│   ├── predictions/ (MakePrediction, MyPredictions)
│   ├── pool/ (PoolDetail, Distribute)
│   └── subscription/ (Plans, CheckoutRedirect, ManageSubscription)
├── components/
│   ├── ui/ (Button, Card, Table, Modal)
│   └── domain/ (MatchCard, PredictionForm, LeaderboardTable, BracketView)
├── router/
│   └── index.tsx             # rutas + guards (protected routes)
└── App.tsx
```

---

3. Puntos clave de la API que moldean el frontend

1. **PUT requiere todos los campos** (no hay PATCH) → los formularios de edición precargan el objeto completo.
2. **Ventana de 30 min antes del partido** para predicciones → deshabilitar el form con countdown, no confiar solo en el 422 del backend.
3. **Errores 422** siguen el shape `{ message, errors: { campo: [...] } }` (Laravel `ValidationException`) → interceptor único que normaliza esto a mensajes por campo en los formularios.
4. **Flujo de pool y bracket son asíncronos** → usar polling (React Query `refetchInterval`) en leaderboard, scoring dashboard y bracket durante partidos en vivo. No hay WebSocket todavía (está en el TODO del backend).
5. **Checkout de Stripe es por redirección**, no modal embebido: `POST /user/subscription/checkout` devuelve una URL y el front hace `window.location.href`.

6. **403 por límites de plan** (`CheckGroupLimits`, `CheckSubscriptionTier`) deben mostrarse como "upgradeá tu plan", no como error genérico. Los torneos con `is_promotional=true` bypasean estos límites — mostrar badge de "acceso gratis por tiempo limitado".
7. **Multiplicador de scoring depende del stage** (`is_knockout`, `stage_order`) → la vista de un match necesita saber en qué stage vive antes de mostrar cuántos puntos vale la predicción.

---

4. Flujos completos del sistema

4.1 Setup

```
Login/registro (Sanctum bearer)
   → Crear torneo + stages (bulk)
   → Crear grupo con scoring/pool configurado
   → Invitar miembros
```

4.2 Predicción y scoring

```
Miembro predice (match scheduled, >30min antes)
   → Match termina (status: finished)
   → Scoring automático (puntos base × multiplicador de stage)
   → Leaderboard se actualiza
```

4.3 Pool de premios

```
Miembros contribuyen (POST /pool, se descuenta bet_amount)
   → Admin distribuye (solo el creador del grupo)
   → Reparto 60/30/10 según leaderboard final
   → Email de notificación a ganadores
```

4.4 Multi-fase / bracket

Configuración:

```
Crear stages (fase de grupos, octavos, cuartos...)
   → Crear tournament-groups (Grupo A, B, C...)
   → Asignar equipos a cada grupo
   → Admin aprueba el grupo (status: approved)
```

Avance entre fases:

```
Definir regla de avance (top_n / best_third / manual)
   → Partidos terminan, standings se actualizan
   → Sistema sugiere avances (POST /stage/{id}/advancements/suggest) → confirmed:
false
   → Admin confirma (PUT /advancement/{id}/confirm o /confirm-all) → confirmed: true
   → Bracket público se actualiza
```

---

5. Pantallas mockeadas

5.1 Bracket público

- Selector de fase (tabs): Fase de grupos, Dieciseisavos, Octavos, Cuartos, Semifinal, Final.
- Vista "Fase de grupos": tarjetas por grupo con tabla de posiciones (posición, equipo, pts, diferencia de gol) + lista compacta de partidos con badge de estado (en vivo / programado / resultado).
- Badge de estado del grupo: "Aprobado" (verde) o "Pendiente" (ámbar) según `tournamentgroups.status`.
- Vista "Eliminación directa": columnas por stage con matches; casillas con borde punteado = "por definir" cuando el avance todavía no está confirmado.
- Nota de multiplicador de fase visible al pie (ej: "Multiplicador de esta fase: 1.5x").

5.2 Admin — aprobación de grupos y confirmación de avances

- **Sección "Grupos pendientes de aprobación"**: lista con nombre de grupo, equipos asignados, botones Aprobar/Rechazar → `POST /tournament-group/{id}/approve` o `/reject`.
- **Sección "Avances sugeridos"**: lista de equipos con su posición y stats, generada tras `POST /stage/{id}/advancements/suggest`. Cada fila tiene botón "Confirmar" individual (`PUT /advancement/{id}/confirm`) y un botón global "Confirmar todos" (`POST /stage/{id}/advancements/confirm-all`).
- Nota visible de qué regla se aplicó (ej: "Regla aplicada: top_n = 2. Uruguay y Chile no avanzan.") para que el admin entienda por qué el sistema sugirió esos equipos.

---

6. Próximos pasos

- [ ] Scaffold del proyecto (Vite + TS + cliente API tipado + auth store + interceptor 422)
- [ ] Pantalla de login funcional contra la API
- [ ] CRUD de torneos y grupos
- [ ] Flujo de predicciones con validación de ventana de 30 min

- [ ] Bracket view + admin de aprobación/avances
- [ ] Integración de Stripe Checkout
- [ ] Fase 2: port de la lógica de negocio a Flutter